

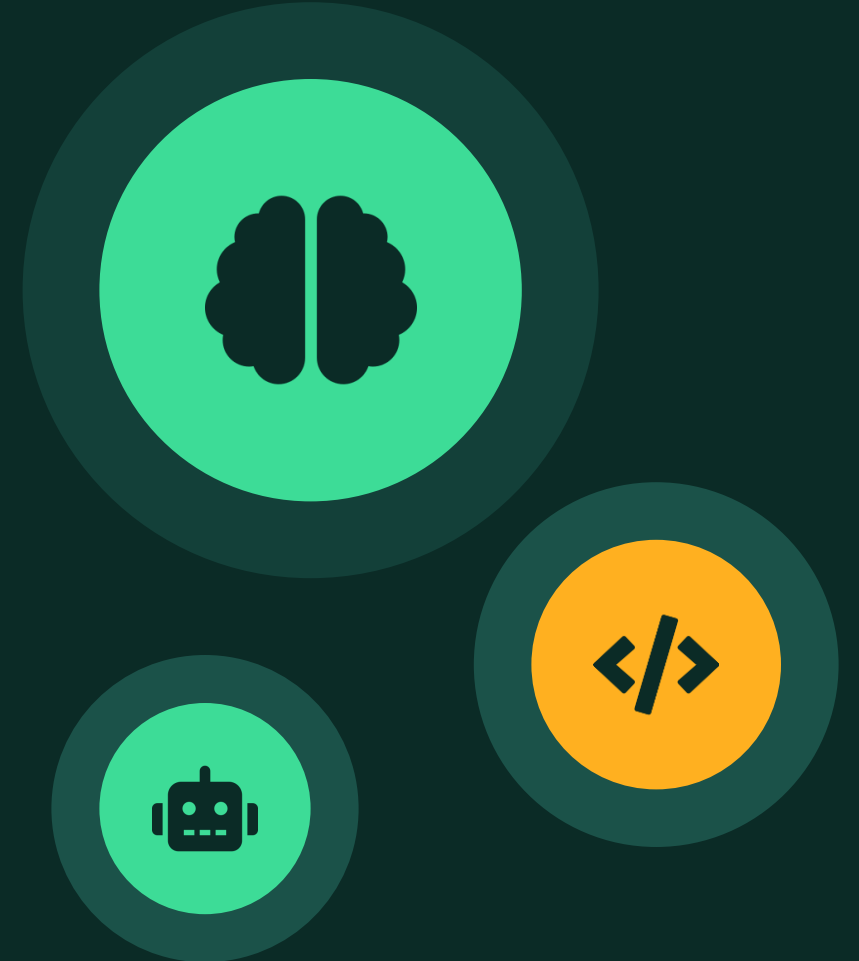
INTRA-COMPANY TRAINING · CGG SERVICES SAS

AI for Developers

Master AI to code faster

Day 2 of 2 · Industrializing AI usage

Trainer: Daouda DIOP



DAY 2

Industrializing AI usage in your development workflow



Warm-up: what did you try since yesterday? What worked, what surprised you?

5 · Agents for complex tasks

6 · AI in your team workflow

7 · Automating the dev cycle

8 · Choosing your AI stack

DAY 2 • MODULE 5

Using AI agents for complex tasks

Multi-file work, large refactoring, advanced tests, automation, multi-agent orchestration, and managing limits.



Multi-file work with agents

1 Interfaces and types

2 Implementations

3 Callers

4 Tests and documentation



Give a map

Which modules are involved.



State the scope and what is out of scope



Ask for a plan across files first



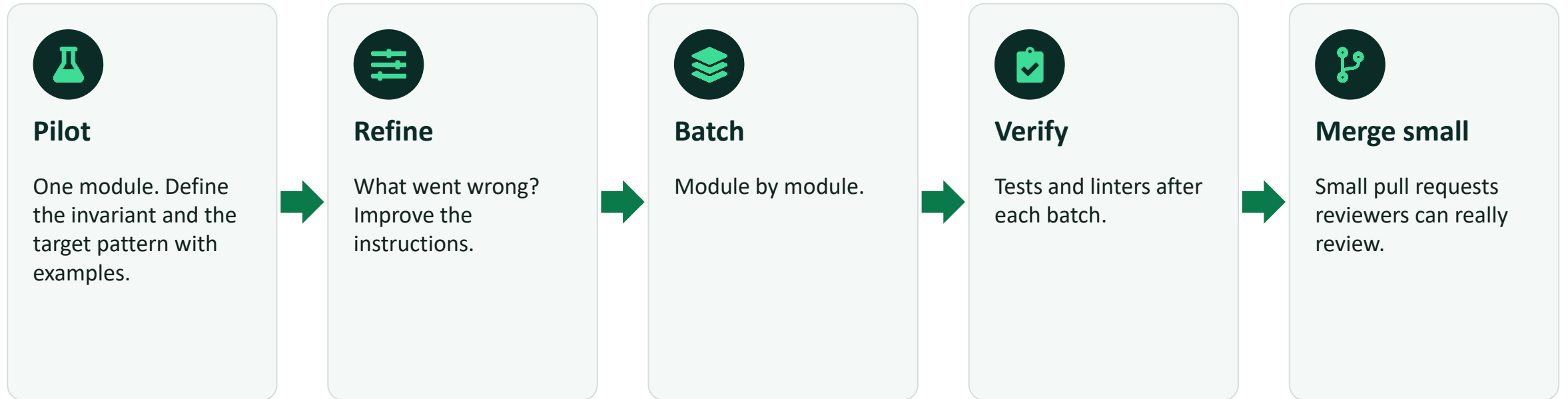
Commit per step



Update tests and docs in the same change

A good order for a change spanning several files

Large-scale refactoring



Mechanical change? Ask the agent to write a transformation script, then review the script instead of hundreds of edited files.

Advanced test generation



Coverage gaps

Give a coverage report; ask for tests on untested branches. Check they are meaningful.



Edge cases and errors

Ask for unusual inputs and failure modes, then test them.



Property-based tests

Where your ecosystem supports them: describe a property that must always hold.



Test strength

Mutation testing: introduce small bugs and check that tests fail.



Coverage measures which lines ran, not whether behavior was checked.

Automating development tasks



Repeatable

It comes back often.



Checkable

Tests, linters or type checks can verify the result.



Low blast radius

If it goes wrong, the damage is small and reversible.

Often a fit

- Code migrations
- Boilerplate, scaffolding
- Lint fixes
- Changelogs
- Docs updates

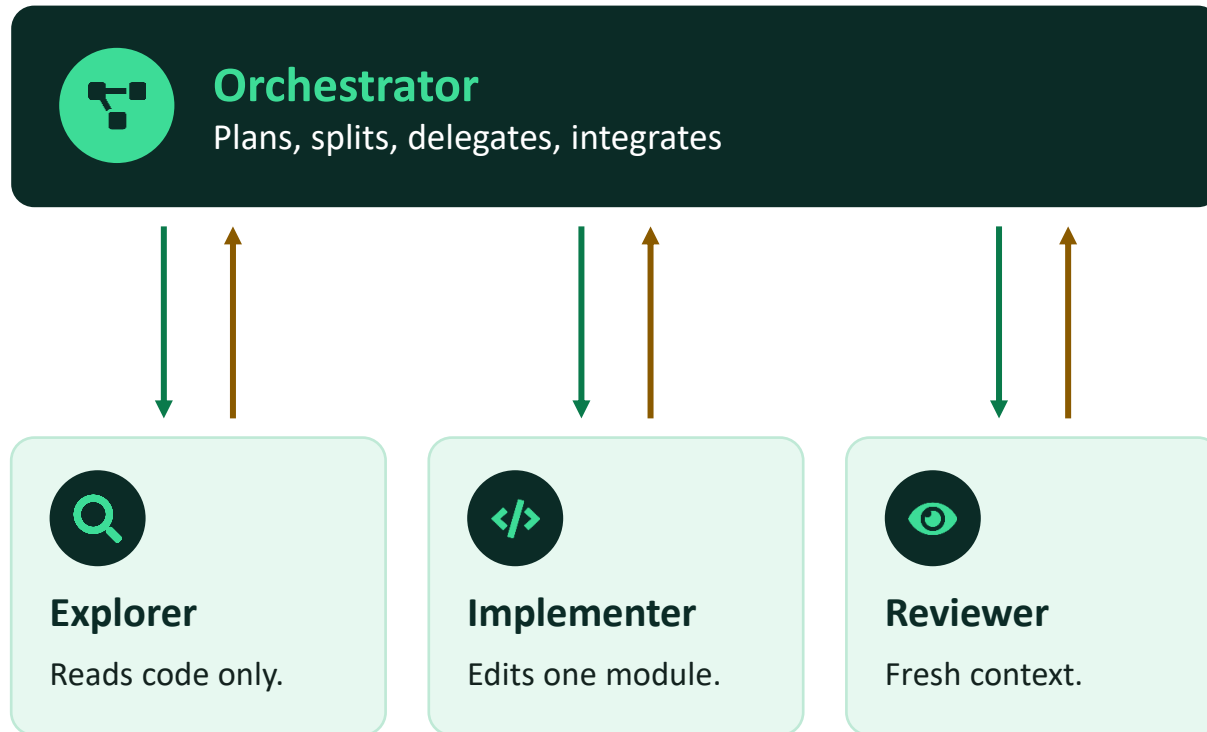
Poor fit

- Ambiguous tasks
- No way to verify
- Production data or secrets



Do it once by hand with the agent, refine the instructions, and only then automate.

Multi-agent orchestration: several agents, one goal



Four common patterns

- 1 Orchestrator + workers**
One agent splits the task and delegates.
- 2 Parallel workers**
Independent subtasks run at the same time.
- 3 Pipeline**
Planner, then implementer, then reviewer.
- 4 Author + independent reviewer**
A second agent checks the first one's work.

 Multi-agent is not a new technology: it is several agent loops, coordinated.

When is multi-agent worth it, and when is it not?



Consider it when...



Context overload

Sub-agents explore and return short summaries.



Independent subtasks

Modules that change in parallel, no shared files.



Different roles or permissions

Read-only explorer, writer, limited reviewer.



Independent verification

A fresh context reviews the work.



Stay with one agent when...



The task is small or well-scoped

Coordination costs more than it saves.



Steps are coupled or sequential

Each step depends on the previous one.



You cannot yet verify the output

More agents means more output to check.



Cost, latency, traceability matter

More tokens, more waiting, harder to debug.



Start with one agent. Add more only when you hit a concrete limit.

Running multi-agent setups safely



Higher cost and latency

Several agents: more tokens, more waiting.



Hand-offs lose information

A sub-agent only knows what it is told. Write a precise brief.



Conflicting edits

Give parallel agents separate branches or working copies.



Errors compound

An early wrong assumption can spread across sub-agents.



Harder to follow

Keep tasks and results explicit. Track who did what.



You still review

Read every diff and run the tests, whoever wrote the code.



A practical start: one orchestrator, two or three narrow sub-agents, clear briefs, and tests as the shared source of truth.

Managing agent limitations

Problem	Countermeasure
Drifts from the goal	Tighter scope, plan first, stop and redirect
Loops on the same failure	Interrupt, restate with what you learned, or start a new session
Context overloaded	Smaller tasks, summaries, restart with notes
Takes risky actions	Approval prompts for sensitive commands, sandbox, work on a branch
Edits tests to make them pass	Protect tests in the instructions, always read the whole diff
Follows instructions hidden in content it reads (prompt injection)	Treat external content as untrusted; minimum permissions
Cost and time	Small scope; lighter models for simple tasks, where possible

Six safety habits when an agent has hands



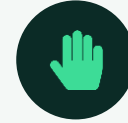
Dedicated branch

Main stays safe.



Least privilege

Only the access it needs. No production credentials.



Approvals on

For risky commands: delete, config, network.



Review every diff

Before every commit.



No secrets

Keep them out of prompts and readable files.



Commit checkpoints

Go back to a good state quickly.

DAY 2 • MODULE 6

Integrating AI into your team workflow

Conventions, code review, documentation, risks and collaboration.



Team conventions for AI usage



Which tools are approved?



What data may, and may not, be shared with them?



When must AI use be mentioned in a pull request?



Who reviews, and who is accountable?



Where do prompts and instruction files live?



What is the quality bar: tests, review?



How do we report an incident?



**A one-page charter beats
a fifty-page policy.**

The discussion matters as much as the
document. Review it regularly.

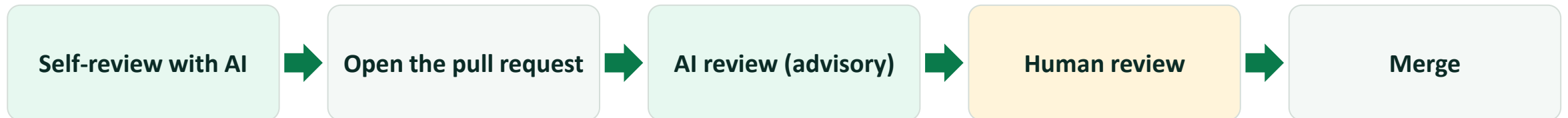
AI-assisted code review

AI as first reviewer

- Typos and obvious bugs
- Missing null checks and tests
- Style inconsistencies
- Summaries of large pull requests

Humans keep

- Design and architecture
- Business rules
- Security judgment
- Maintainability and intent



AI review supplements human review; it does not replace it.

Automatically generated documentation



README and guides

Setup, usage, onboarding for new colleagues.



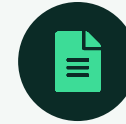
Docstrings and API docs

Generated from the real code.



Architecture notes

Drafts of decision records.



Release notes

Changelogs from commits and pull requests.

1

Generate from the code

Give the tool the real files.

2

Review for accuracy

A wrong document is worse than none.

3

Keep it in the repo

It changes with the code.

4

Update when the code changes

Make it part of the workflow.

Risk management for AI-generated code

Risk	Mitigation
Confidential data leakage	Approved tools only; clear data rules; no secrets in prompts
Vulnerabilities in generated code	Security review, static analysis and tests, as for human code
Licensing and intellectual property	Understand tool terms and company policy; involve legal where needed
Hallucinated dependencies	Verify each new dependency exists, is maintained and is the right one
Over-reliance, loss of skills	Keep reviewing and understanding; keep practicing core skills
Compliance and auditability	Keep traceability of AI use; align with company requirements

Human-AI collaboration: who does what?



You bring

- Intent: what problem, and why
- Judgment: what is good enough
- Accountability: your name is on the commit
- Domain knowledge
- Trade-offs: speed, cost, risk, maintainability



The AI brings

- Speed
- Wide knowledge of patterns
- Tireless drafting
- A first pass on many tasks
- Summaries and translation between formats



The AI drafts. Humans decide.

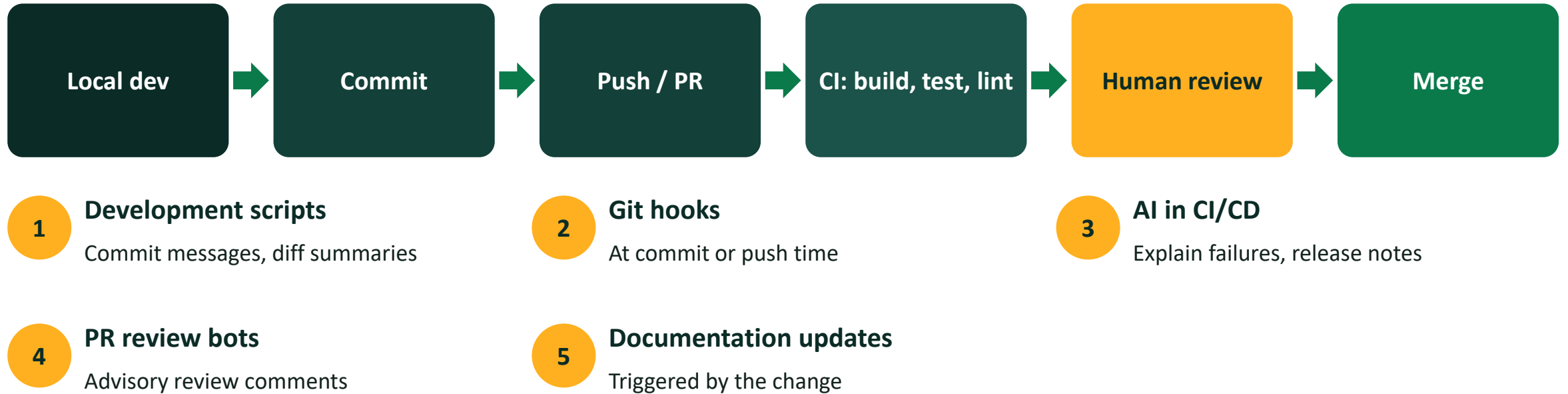
DAY 2 • MODULE 7

Automating the development cycle with AI

Scripts, hooks, CI/CD and pull request review bots.



Where AI can plug into your pipeline



The deterministic gates stay: tests, lint, human review. AI is an extra signal, not a replacement.

AI-assisted development scripts

- **Good candidates:** commit message from staged diff, diff summary, CI log explanation, release notes, scaffolding.
- **Send the minimum context.** The diff, not the repository.
- **Log inputs and outputs.** So you can debug.
- **Fail gracefully.** Never block the developer silently.
- **Suggest, do not act.** Keep the human in the loop.

PSEUDO-CODE

```
#!/usr/bin/env bash
# Pseudo-code: adapt to your approved tool
DIFF=$(git diff --staged)

MSG=$(echo "$DIFF" | your-ai-cli \
  --prompt "Write a conventional commit
  message for this diff. Title max 72 chars.")

echo "Suggested commit message:"
echo "$MSG"
# The developer decides whether to use it.
```

Git hooks with AI

Hook moment	What AI can do	Caution
Commit message	Suggest a message from the staged diff	A suggestion; the developer edits and decides
Before commit or push	Quick review of the staged changes	Must be fast and must tolerate failure
Before commit	Flag likely mistakes, such as a committed secret	Prefer deterministic scanners for hard blocking



Fast

A slow hook gets bypassed.



Fail-safe

If the AI is unreachable, do not block.



Advisory

Deterministic checks block; AI checks advise.



Data rules

Same rules on what data is sent out.

AI in CI/CD

Good uses

- Review comments on pull requests
- Suggestions for missing tests
- Failure triage in plain language
- Changelogs and release notes
- Checks that docs match the change

1

Advisory before blocking

2

Deterministic checks stay the gate

3

Secrets in the CI store; minimum permissions

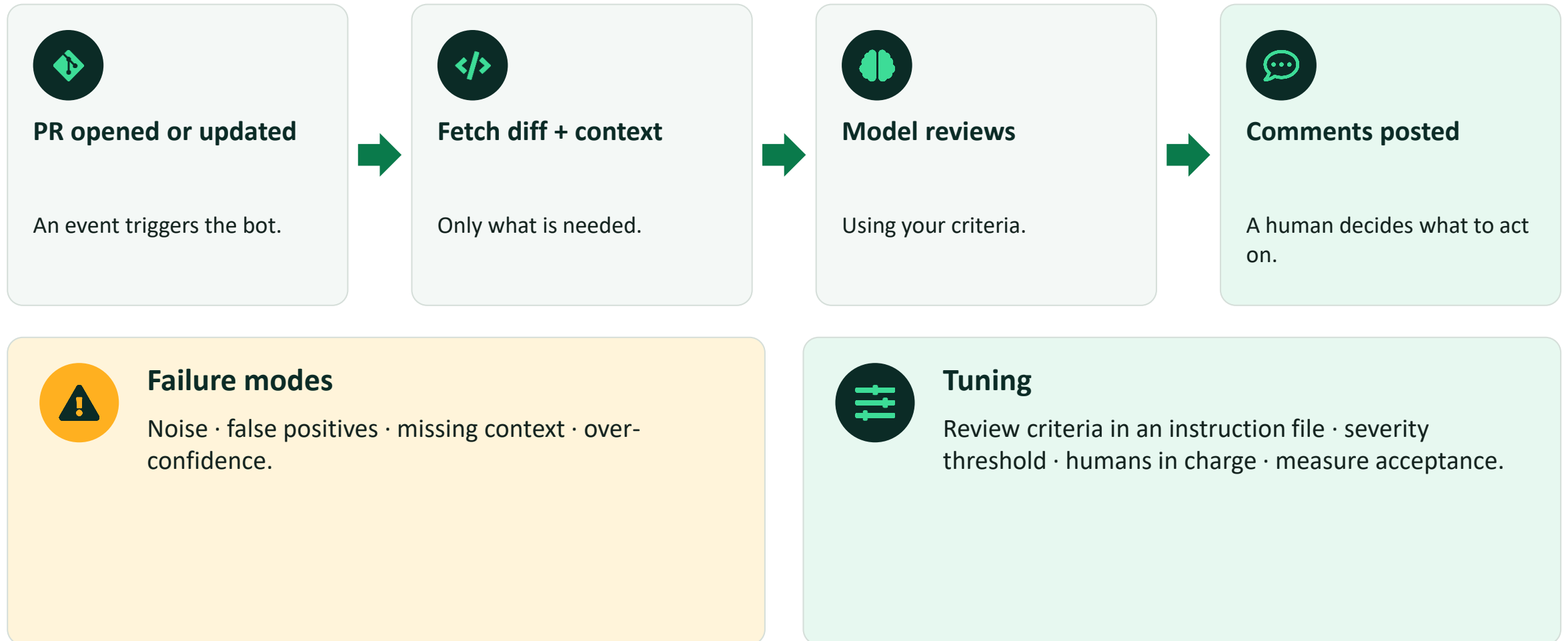
4

Treat pull request text as untrusted

5

Send only what is necessary; watch the cost

Pull request review bots



DAY 2 • MODULE 8

Choosing your AI development stack

Agents vs AI-assisted IDEs, decision criteria and standardization.



Agents vs AI-assisted IDEs, compared

Dimension	AI-assisted IDE	Autonomous agent
Best for	Work in flow; visible, immediate edits	Multi-step, multi-file tasks
Control	You approve each change as it appears	You review a result after several steps (configurable)
Context	Open files and workspace	Explores the repository itself
Autonomy	Lower	Higher
Learning curve	Familiar: lives in your editor	New habit: give a goal, review a result
Risk profile	Mostly small, visible changes	Larger changes: guardrails matter more
Team fit	Daily flow	Bigger tasks. Many teams use both

Typical tendencies, not absolute truths. The two families are converging.

Choosing by project type, team size and security

Situation	Leaning
Large legacy codebase	Agent for exploration; IDE assistant for edits
Small greenfield prototype	Either; an agent can be fast for scaffolding
Small team	Lightweight and flexible, with shared conventions
Larger team or several teams	Standardize on one or two approved tools, central configuration
Strict confidentiality	Approved options only; check data flows first (private or local options, depending on policy)
Regulated or critical software	Stronger review gates and a clear audit trail



Whatever the situation: run a small pilot first, on a real but low-risk task, and decide with evidence.

Standardizing tools across the team



Standardize

- The list of approved tools
- Shared instruction files in the repo
- A shared prompt library
- A shared review checklist
- An onboarding guide
- A regular review of your choices

Leave free

- Personal editor preferences
- Themes and shortcuts

Standardize inputs and outputs, not the personal workflow.

Hands-on: industrialize the project with AI



Generate automated tests with AI



Structure the project: README, docs, conventions



Automate the development workflow



Set up AI-assisted code review



Build a complete AI workflow for developers

SUGGESTED DELIVERABLES

- A project instruction file
- 2–3 prompt templates
- A review checklist
- A draft CI or hook integration

Safety habits: branch, least privilege, review every diff, no secrets.

Ten rules to take away

1

An LLM predicts tokens. Plausible is not true.

2

Context is everything. Relevant beats big.

3

Agent = model + tools + loop. The harness acts.

4

A prompt is a specification: six blocks.

5

Small steps, tests and diff review.

6

Match the tool to the job and the security needs.

7

You own every line you ship.

8

Team knowledge lives in the repo.

9

Automate gradually: advisory first.

10

Re-evaluate regularly: the field moves fast.

Your action plan



This week: for yourself

Use the six-block prompt on a real task.
Write an instruction file for your main repo.
Apply the five checks before accepting code.



Next weeks: with your team

Draft a one-page AI charter.
Share 2–3 prompt templates in the repo.
Agree on approved tools and data rules.



Later: scale carefully

Try an advisory AI review on pull requests or CI, and measure it.
Schedule regular reviews of your tools and practices.



Name one action you will take next week. Write it in the chat.

Evaluation, certificate and follow-up



Evaluation

Based on the practical work you shared with me in real time.



Training report

Sent by email to each participant, with the trainer's assessment.



Certificate of success

Sent by email, to share if you wish.

Optional follow-up

- Regular phone meetings to answer your questions
- On-site presence at the start of your project
- An audit of your practices a few weeks or months later

THANK YOU, CGG SERVICES SAS

Questions?

Understand it. Give it context. Verify it. Own it.

AI for Developers · Master AI to code faster

Human Coders • OptiData Services • Trainer: Daouda DIOP

